Applied Machine Learning
Unit 04 — Lecture 05 Notes
Data Cleaning: Missing Data, Outliers and Transformation

Tofik Ali

Autumn 2026

Read this first

These notes are written so that you can learn the material *without* having been in the lecture. Every concept is developed from scratch, every number you see was produced by running the code shown beside it, and every figure was generated from real computation rather than drawn by hand.

Units I to III built a model, a loss and five ways to descend it. All of that assumed a clean numeric matrix with no holes in it. No real dataset is like that. Values go missing, sensors stick, a column has one long tail, and somebody types 200 into an age field. This lecture is about what you do before the model, which in practice is where most of the work — and most of the damage — happens.

Four questions organise it. What do I do about a **hole**? About a value that looks **impossible**? About a variable whose distribution is **lopsided**? And, quietly the most important, **in what order** do I do these things relative to the train/test split?

The single result to carry out of the lecture: on thirty perfectly good sensor readings contaminated with five obviously wrong ones, the $|z| > 3$ rule — the one everybody is taught — flags **none of them**. Work through Section 4 with a Python session open until you can say exactly why.

Contents

1 How to use these notes

Read in order: Section 4 uses the skewness idea that Section 5 develops, and Section 6 only makes sense once you know what the earlier steps are. Everything here assumes very little from Units I–III — a train/test split, a fitted model and a score — and no optimisation theory at all.

There are four kinds of box. **Key idea** — the one sentence to remember a month from now. **Worked example** — a calculation done by hand, with small numbers, before any library is used. **Caution** — the specific mistake that section exists to prevent. **Try it yourself** — something to run, with the expected output given so you can check.

Code appears in a framed block, and directly beneath it a shaded block showing what that code *actually printed*. If you run it and get something different, trust your machine and tell me.

1.1 What you need

```
pip install numpy scipy scikit-learn matplotlib pandas
```

Every number in these notes was produced with:

```
import numpy, scipy, sklearn, matplotlib
print("numpy", numpy.__version__, "| scipy", scipy.__version__,
      "| sklearn", sklearn.__version__,
      "| matplotlib", matplotlib.__version__)
```

```
numpy 2.2.6 | scipy 1.15.3 | sklearn 1.7.2 | matplotlib 3.10.9
```

Two real datasets recur, and **neither downloads anything**. Both are bundled inside scikit-learn itself.

```
import numpy as np
from scipy import stats
from sklearn.datasets import load_diabetes, load_breast_cancer

dia = load_diabetes()
Xd, yd = dia.data, dia.target.astype(float)
Xr = load_diabetes(scaled=False).data # the same table in original units
bc = load_breast_cancer()
Xb, yb = bc.data, bc.target
```

```
load_diabetes      442 rows x 10 features, target continuous
load_breast_cancer 569 rows x 30 features, target 0/1
missing cells in either table:  0
baseline 5-fold accuracy, breast_cancer  0.9807  +/- 0.0065
baseline 5-fold R2,       diabetes       0.4823  +/- 0.0493
```

Both tables are perfectly clean, which is exactly why they are useful: we break them ourselves, in controlled ways, so we always know the right answer.

Two datasets, one seed — the convention for this lecture

Everything in these notes is computed from the **two bundled tables** above and **one seed**. The seed is **0**: every generator that shuffles, splits, punches holes or drives a stochastic estimator is created as `np.random.default_rng(0)` or given `random_state=0`, and every listing below that involves chance shows that line rather than hiding it in a helper.
Two things are deliberately not that seed, and it is worth knowing why. The 35 sensor readings used throughout the outlier section come from `np.random.default_rng(7)`: that

number *defines the data* the section is about, not the experiment run on it, so it stays where it is. And the repeated-split experiments sweep `random_state` over a whole range — `range(0, 20)`, `range(0, 300)` — because the spread *across* splits is the thing being measured. Collapsing those to one seed would delete the result.
So every number printed in these notes reproduces *exactly* if you run the code as written. If you get something different, the difference is in your code or your library versions, not in chance — which is the only thing that makes a printed number worth reading.

Do not reach for `fetch_*`

Every `fetch_*` loader downloads over HTTPS, and on the campus network those downloads fail with a certificate error you cannot fix from your machine. Nothing in this course depends on one: use the bundled loaders (`load_diabetes`, `load_breast_cancer`, `load_iris`, `load_wine`, `load_digits`) or a fixed-seed synthetic set.

1.2 Learning outcomes

By the end of this lecture you should be able to:

1. Describe the stages of data preprocessing and explain why data cleaning precedes every other step in a machine-learning workflow (CO1).
2. Distinguish MCAR, MAR and MNAR missingness, say what each implies for complete-case analysis, and predict which of them an imputer can and cannot repair (CO1, CO2).
3. Apply mean, median, mode, k-nearest-neighbour and iterative imputation; derive the exact factor by which mean imputation shrinks a variance, and quantify the effect of each method on a distribution and on a downstream model's score (CO2).
4. Detect outliers using the z-score, the interquartile-range fence and the modified z-score; explain masking, and state the sample-size ceiling on $|z|$ (CO1, CO2).
5. Apply numerical methods for handling outliers — binning and smoothing, trimming, winsorising, capping, robust scaling — and log, square-root, Box–Cox and Yeo–Johnson transformations to a skewed variable, choosing among them with a stated reason (CO2).
6. Order every cleaning step correctly with respect to the train/test split, and express the whole sequence as a scikit-learn `Pipeline` (CO2).

2 Data preprocessing: what it is and why it comes first

2.1 The four tasks, and what "dirty" means

Han, Kamber and Pei divide preprocessing into four tasks, and the division maps onto the next three lectures. **Data cleaning** — fill in missing values, smooth noise, identify outliers, resolve inconsistencies — is this lecture. **Data integration** merges tables from several sources. **Data reduction** produces fewer rows or columns with as little loss as possible, and is the next lecture. **Data transformation** puts values on a scale or in a shape the downstream method can use, and is split between this lecture (shape) and the next (scale).

Cleaning comes first for an unglamorous reason: everything downstream inherits its mistakes silently. A model does not warn you that a quarter of one column is a constant you invented. It fits, it reports a number, and the number looks fine.

Defect	What it looks like in the file
incomplete	a blank cell, an `NaN`, a -1 or a 9999 that means "not asked"
noisy	a value that is wrong by a little — rounding, sensor jitter, a transcription slip
outlying	a value that is wrong by a lot, *or* right and rare
skewed	a long tail, so that the mean is not a typical value
inconsistent	`"M"`, `"Male"`, `"male"`, `1` for the same thing
duplicated	the same patient entered twice under two identifiers

The first four are this lecture. The last two are handled with the encoding material in the next one.

2.2 The default fix, and what it costs

The instinctive response to a hole is to delete the row that contains it. Statisticians call this **listwise deletion** or **complete-case analysis**, and pandas spells it `df.dropna()`. It is one line, it is honest in the sense that it invents nothing, and on a wide table it is a disaster.

Why deletion is exponential in the number of columns

Suppose each cell is independently missing with probability p, and there are d columns. A row survives only if all d of its cells survive, so

$$P(\text{row survives}) = (1-p)^d. \tag{1}$$

That is an exponential in d, and nobody's intuition is exponential. With $p = 5\%$:

	$d = 5$	$d = 30$	$d = 50$
$p = 0.01$	0.951	0.740	0.605
$p = 0.05$	0.774	0.215	0.077
$p = 0.10$	0.590	0.042	0.005

At a per-cell rate of five per cent you lose half your rows once you have $\log 0.5 / \log 0.95 = 13.5$ columns. Thirty columns and you keep one row in five.

Now do it for real. Knock 5% of the cells out of `load_breast_cancer` at random and count.

```
rng = np.random.default_rng(0)
Xb_miss = Xb.copy()
holes = rng.random(Xb.shape) < 0.05 # 5% of CELLS, completely at random
Xb_miss[holes] = np.nan
complete = ~np.isnan(Xb_miss).any(axis=1)
print("rows left after listwise deletion", complete.sum())
```

```
fraction of individual cells missing      0.051
rows with at least one missing value      455 of 569
rows left after listwise deletion         114  (20.0%)
predicted survival rate  0.95**30       = 0.215
```

Measured 20.0% against a predicted 21.5%; the difference is one draw of sampling noise. **Five per cent of the cells cost eighty per cent of the patients.**

Was it at least safe? On this dataset, more or less — and it is worth being honest about that rather than overstating the case.

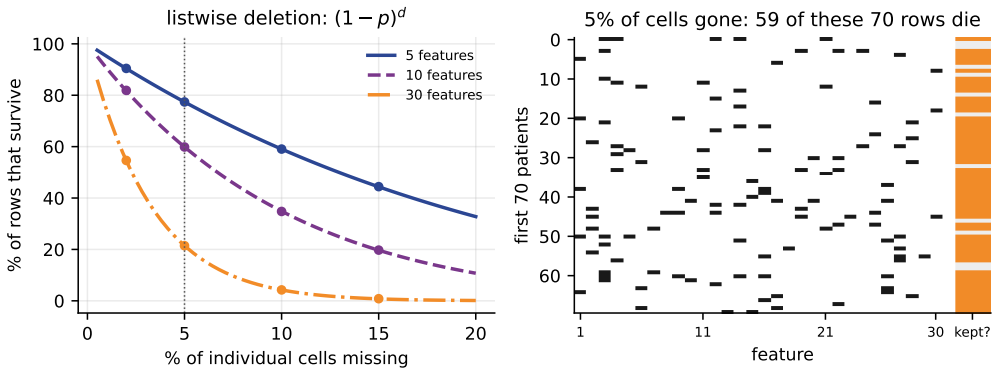

Figure 1: Left: Equation (1), drawn, with measured points from 200 simulations each. The dotted line is $p = 5\%$. Right: the actual missingness mask for the first 70 patients. Black cells are missing; the strip on the right is orange if that row is deleted. Fifty-nine of these seventy patients die of one missing number.

```
accuracy on the 114 survivors              0.9735  +/- 0.0216
accuracy imputing instead, all 569 rows    0.9719  +/- 0.0129
baseline with no missingness at all        0.9807  +/- 0.0065
```

Deleting did not lose accuracy here. It lost 455 patients and produced an estimate with 1.7 times the spread. On a harder problem it is not so kind: Section 3.5 shows the same procedure on `load_diabetes` at 30% missingness leaving **nine** training rows for ten features and a model scoring $R^2 = -4.70$.

Key idea

Deleting incomplete rows is a decision to throw away evidence, and the amount you throw away grows exponentially with the number of columns. Sometimes it is the right decision. It is never the free one.

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 3.;Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 3–4.;James, Witten, Hastie, Tibshirani and Taylor, *An Introduction to Statistical Learning with Python*, Ch. 5–6, 12.;Scikit-learn documentation (preprocessing, model selection), accessed 2026-08-04.

3 Handling missing data

3.1 Three questions to ask about a hole

Before you type `fillna`, answer these.

1. Why is it missing? A sensor failed, a question was refused, a test was never ordered, a field was added to the form in year three — four different problems. **2. Is the reason related to the value that is missing?** This is the MCAR / MAR / MNAR question of Section 3.4, and it decides whether any imputer can help you at all. **3. Is the *fact* of the hole informative?** If a test is ordered only for patients who already look ill, then "this value is missing" predicts the label; Section 3.6 measures that.

3.2 Mean, median and mode, by hand

Eight marks, two of them missing

$$55,\ 62,\ ?,\ 71,\ 58,\ ?,\ 90,\ 64$$

Six values are observed. Sort them: 55, 58, 62, 64, 71, 90.

Mean. $(55 + 62 + 71 + 58 + 90 + 64)/6 = 400/6 = 66.6667$.
Median. With six values it is the average of the third and fourth: $(62 + 64)/2 = 63$.
Standard deviation, using the $n - 1$ divisor: $s = \sqrt{\frac{1}{5}\sum(x_i - 66.6667)^2} = 12.6754$.

The mean sits 3.67 marks above the median because of the single 90. That gap is the whole reason both statistics exist: the mean uses every value's distance, the median only its rank.

Which do you fill with? *Mean* for a roughly symmetric numeric column. *Median* when the column is skewed or has outliers, because the median does not move when a far-away point moves further away. *Mode* — the most frequent category — for a categorical column, where a mean is not even defined.

```
marks = np.array([55, 62, np.nan, 71, 58, np.nan, 90, 64])
obs = marks[~np.isnan(marks)]
for name, fill in (("mean", obs.mean()), ("median", np.median(obs))):
    f = np.where(np.isnan(marks), fill, marks)
    print(name, f.mean(), f.std(ddof=1))
```

```
observed values       [55, 62, 71, 58, 90, 64]
n observed 6 of 8
mean of observed         66.6667
median of observed       63.0000
sd of observed (ddof=1)  12.6754
mean   imputed:  mean 66.6667   sd 10.7127   sd is now 84.5% of the truth
median imputed:  mean 65.7500   sd 10.8463   sd is now 85.6% of the truth
```

Two things happened that nobody asked for. The mean-imputed column kept its mean — of course, you added copies of the mean — but its standard deviation fell by 15.5%. The median-imputed column's mean *moved*, from 66.67 to 65.75.

3.3 Mean imputation shrinks the variance, exactly

That 15.5% is not an accident of these eight numbers. It is a theorem.

The shrinkage factor

Let m values be observed out of n, with observed mean $\bar{x}$. Fill the $n - m$ holes with $\bar{x}$.
The mean cannot move. The new sum is $\sum_{i=1}^{m} x_i + (n - m)\bar{x} = m\bar{x} + (n - m)\bar{x} = n\bar{x}$, so the new mean is $\bar{x}$.
The sum of squared deviations cannot move either. Each new value contributes $(\bar{x} - \bar{x})^2 = 0$.
So the numerator of the variance is unchanged while the denominator grows:

$$\sigma^2_{\text{after}} = \frac{\sum_{i=1}^{m}(x_i - \bar{x})^2}{n} = \frac{m}{n}\,\sigma^2_{\text{before}}, \qquad \frac{\sigma_{\text{after}}}{\sigma_{\text{before}}} = \sqrt{\frac{m}{n}} = \sqrt{1 - p}, \tag{2}$$

where $p = 1 - m/n$ is the fraction imputed. With the $n - 1$ divisor the corresponding factor is $(m - 1)/(n - 1)$.
For the eight marks: $m = 6$, $n = 8$, so the population variance is multiplied by $6/8 = 0.75$ and the sample variance by $5/7 = 0.7143$. The standard deviation ratio is $\sqrt{0.7143} = 0.8452$,

and $10.7127/12.6754 = 0.8452$.

```
mean imputation and the variance, exactly
  population variance ratio  m/n         = 0.7500
  sample     variance ratio  (m-1)/(n-1) = 0.7143
  measured, population (ddof=0)          = 0.7500
  measured, sample     (ddof=1)          = 0.7143
```

Now on a real column. Body mass index for the 442 diabetes patients, in $\mathrm{kg/m^2}$, with the true standard deviation 4.4181.

```
col = Xr[:, 2] # BMI, original units
rng = np.random.default_rng(0) # the one seed for this lecture
for p in (0.1, 0.3, 0.5, 0.7):
    r = []
    for _ in range(2000):
        m = rng.random(len(col)) < p
        v = col.copy()
        v[m] = v[~m].mean()
        r.append(v.std(ddof=1) / col.std(ddof=1))
    print(p, np.mean(r), np.sqrt(1 - p))
```

```
the same law on a real column: diabetes BMI, 2000 draws per row
  p=0.1   measured sd ratio 0.9486   sqrt(1-p) = 0.9487
  p=0.3   measured sd ratio 0.8363   sqrt(1-p) = 0.8367
  p=0.5   measured sd ratio 0.7059   sqrt(1-p) = 0.7071
  p=0.7   measured sd ratio 0.5447   sqrt(1-p) = 0.5477
  largest gap between measurement and sqrt(1-p) here  0.0031
```

Two thousand repetitions per point, and the largest disagreement with $\sqrt{1-p}$ across the four rows is 0.0031. Why care? Because everything you compute afterwards is built out of that standard deviation.

```
and what that does to a correlation: BMI against the target
  complete data       corr(BMI, y) = 0.5865
  40% mean-imputed    corr(BMI, y) = 0.4538  (77% of it survives)
```

Mean imputation manufactures confidence you do not have

It does not merely lose information — it *understates* the spread of the data, so every downstream standard error is too small, every confidence interval too narrow and every correlation attenuated. A quarter of the BMI–target correlation disappeared at 40% imputation. Nothing in any library's output warns you about this, because from the library's point of view nothing went wrong.

3.4 MCAR, MAR and MNAR

Write R for the indicator that a value is missing, X_{obs} for the data you can see and X_{mis} for the values you cannot. Rubin's taxonomy asks a single question: *what does $P(R)$ depend on?*

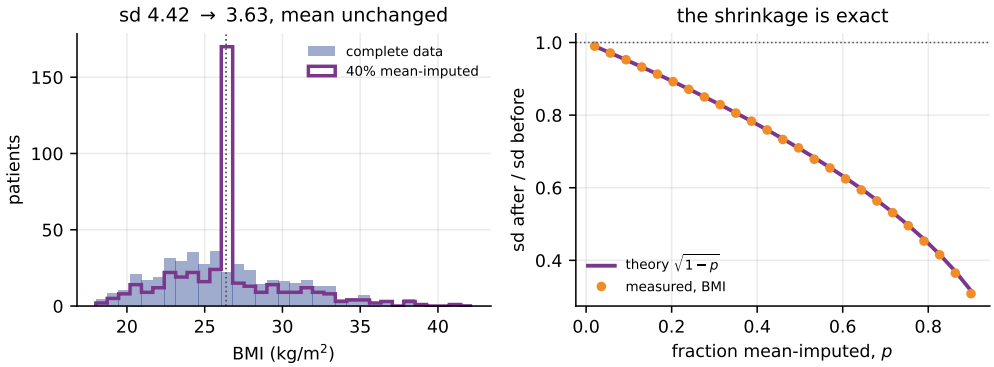

Figure 2: Left: the BMI column before and after mean-imputing it at $p = 0.40$. Mean imputation does not blur the distribution — it piles every hidden patient onto one value, and the standard deviation falls from 4.4181 to 3.6309. This single draw hid 35.29% of the column rather than exactly 40%, which is why the drop is smaller than $\sqrt{0.6}$ would suggest; against the 286 values still observed the shrinkage is exact, $0.803902 = \sqrt{285/441}$. Right: the average over 400 draws per point against $\sqrt{1-p}$, which never disagrees by more than 0.0080.

Name	$P(R)$ **depends on**	**Example**
MCAR	nothing	a tube was dropped in the lab; a page of the questionnaire was lost
MAR	X_{obs} only	BMI was recorded only for patients flagged with high blood pressure — and blood pressure *is* in the table
MNAR	X_{mis} itself	the heaviest patients declined to be weighed; the highest earners refused the income question

The names are unhelpful — "missing at random" does not mean random — but the consequences are sharp. Under **MCAR** the complete cases are a fair random sample of the whole: you lose statistical power, not correctness. Under **MAR** the complete cases are biased, but the variables that caused the bias are in your table, so a model that conditions on them can undo it. Under **MNAR** the information you would need was never recorded, and *no procedure applied to the data can fix this* — you need an external assumption, a sensitivity analysis, or better data collection.

Construct all three on the same column and measure.

```
bmi, bp = Xr[:, 2], Xr[:, 3]
rng = np.random.default_rng(0) # the one seed for this lecture
mcar = rng.random(n) < 0.40 # a coin
mar = bp < np.median(bp) # depends on BP, which we see
mnar = bmi > np.quantile(bmi, 0.60) # depends on BMI itself
for name, m in (("MCAR", mcar), ("MAR ", mar), ("MNAR", mnar)):
    print(name, bmi[~m].mean(), bmi[~m].mean() - bmi.mean())
```

```
complete-data mean BMI  26.3758 kg/m2   (n = 442)
  MCAR  35.3% missing   complete-case mean 26.4913   bias +0.1155
  MAR   48.0% missing   complete-case mean 27.7230   bias +1.3473
  MNAR  40.0% missing   complete-case mean 23.4253   bias -2.9505
```

Throwing away the incomplete rows costs +0.12 kg/m^2 under MCAR — noise, and it would change sign with another draw — against +1.35 under MAR and −2.95 under MNAR. The last

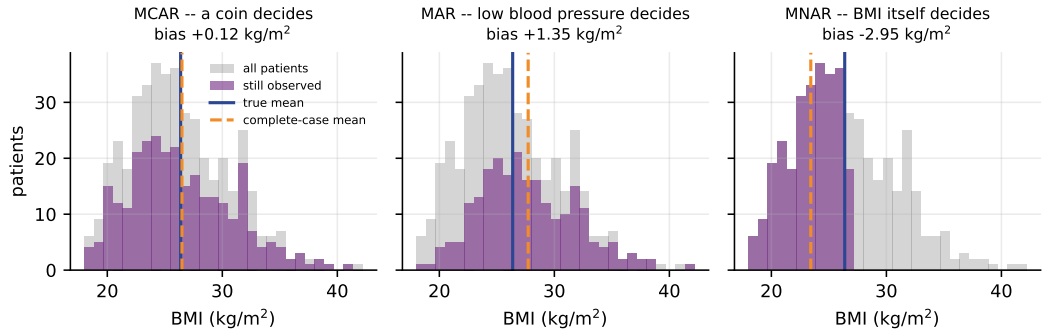

Figure 3: Grey is every patient, purple is who is still observed. Under MCAR the purple histogram is a shrunken copy of the grey one and the two vertical lines coincide. Under MNAR the right tail is simply gone.

of those is two thirds of a standard deviation, from a decision nobody consciously made.

Now the question that makes the taxonomy operational: can a smarter imputer repair the damage? `IterativeImputer` — scikit-learn's version of MICE — models each incomplete column as a regression on all the others and cycles until it settles.

```
from sklearn.experimental import enable_iterative_imputer # required
from sklearn.impute import SimpleImputer, IterativeImputer
for name, m in mech.items():
    Z = Xr.copy(); Z[m, 2] = np.nan
    for label, imp in (("mean", SimpleImputer(strategy="mean")),
                       ("MICE", IterativeImputer(random_state=0,
                                                 max_iter=30))):
        print(name, label, imp.fit_transform(Z)[:, 2].mean())
```

```
can an imputer that uses the other columns repair it?
  MCAR mean     imputed mean 26.4913   bias +0.1155
  MCAR MICE     imputed mean 26.4763   bias +0.1005
  MAR  mean     imputed mean 27.7230   bias +1.3473
  MAR  MICE     imputed mean 26.2170   bias -0.1588
  MNAR mean     imputed mean 23.4253   bias -2.9505
  MNAR MICE     imputed mean 23.7924   bias -2.5834
```

Read the middle pair. Under MAR, MICE cut the bias from $+1.3473$ to -0.1588 — a factor of 8.5 — because blood pressure was recorded and BMI can be predicted from it. Read the last pair. Under MNAR it moved -2.9505 to -2.5834 and stopped, because the patients it needed to learn from are not in the file.

Key idea

The mechanism decides what is possible. MCAR: lose power, keep correctness. MAR: a multivariate imputer can recover the truth, and here it recovered 88% of the bias. MNAR: nothing in the data can help, and pretending otherwise is how a wrong number gets published.

It gets worse than a shifted mean. Fit a linear regression of disease progression on all ten features and read off the BMI coefficient.

```
and what the MNAR column does to a fitted coefficient
  true BMI coefficient             5.603
  MNAR + mean imputation          -0.413
  MNAR + MICE                      2.299
```

The sign flipped

Mean-imputing an MNAR column replaced the entire upper 40% of the BMI distribution with a single constant. The slope fitted through what remained points the *other way*: +5.603 became −0.413. Written up, that reads "BMI is not associated with disease progression", which is the opposite of the truth. Prediction is more forgiving than inference, but only because nobody reads a predictor's coefficients.

3.5 So which imputer should you actually use?

The honest experiment: hold out a clean test set, punch MCAR holes in the *training* matrix only, and compare seven strategies over twenty random splits.

```
for seed in range(0, 20): # a sweep: the spread over splits is the point
    Xtr, Xte, ytr, yte = train_test_split(Xd, yd, test_size=0.25,
                                          random_state=seed)
    Xm = Xtr.copy()
    Xm[np.random.default_rng(100 + seed).random(Xtr.shape) < p] = np.nan
    mdl = make_pipeline(imputer, LinearRegression()).fit(Xm, ytr)
    score = mdl.score(Xte, yte)
```

```
diabetes, 10% of TRAIN cells missing (MCAR), clean test set, 20 splits
  oracle, no missingness   test R2 0.4762  +/- 0.0506
  drop incomplete rows     test R2 0.4264  +/- 0.0630
  mean                     test R2 0.4721  +/- 0.0548
  median                   test R2 0.4727  +/- 0.0544
  constant 0               test R2 0.4723  +/- 0.0548
  KNN, k=5                 test R2 0.4743  +/- 0.0527
  iterative (MICE)         test R2 0.4751  +/- 0.0505
  rows surviving listwise deletion 114 of 331   (1-p)**10 = 0.349
```

```
diabetes, 30% of TRAIN cells missing (MCAR), clean test set, 20 splits
  oracle, no missingness   test R2 0.4762  +/- 0.0506
  drop incomplete rows     test R2 -4.6952  +/- 7.8838
  mean                     test R2 0.4351  +/- 0.0697
  median                   test R2 0.4364  +/- 0.0698
  constant 0               test R2 0.4365  +/- 0.0690
  KNN, k=5                 test R2 0.4591  +/- 0.0580
  iterative (MICE)         test R2 0.2639  +/- 0.5004
  rows surviving listwise deletion 9 of 331   (1-p)**10 = 0.028
```

Three readings, in order of importance.

Dropping rows is expensive and everything else is cheap. At 10% missingness the five imputers span 0.4721 to 0.4751 — three thousandths of R^2 — and all sit within 0.004 of the oracle that never lost any data. Deletion costs 0.05. At 30% it leaves nine rows for ten features and the fit is worse than predicting the mean.

At heavy missingness a multivariate imputer earns its keep: KNN at 30% recovers 0.4591 against mean imputation's 0.4351.

And be honest about MICE. Its 30% entry is 0.2639 ± 0.5004 — a standard deviation ten times any other row's — because on two of the twenty splits the round-robin regressions produced nearly collinear imputed columns and the least-squares fit blew up. Bounding with `min_value` and `max_value` reduced this but did not eliminate it.

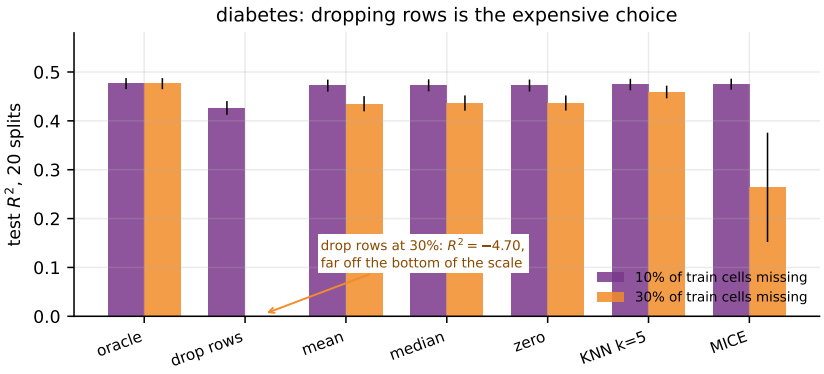

Figure 4: The two tables above, drawn. Error bars are standard errors over twenty splits. Six of the seven bars are the same height; the one that is not is the one most people reach for first.

Key idea

Do not agonise over which imputer. Do agonise over whether you are deleting rows. On these measurements the gap between the best and worst imputer was 0.003 of R^2 and the gap between imputing and deleting was 0.05 — and at 30% missingness, 5.1.

3.6 Missingness is itself data

Sometimes the most informative thing about a value is that it is not there. A diagnostic test that is only ordered when a clinician already suspects something will be missing for the healthy patients. Then "missing" predicts the label directly, and imputing it away destroys that.

```
SimpleImputer(strategy="mean", add_indicator=True)
```

```
one weak feature, missing 80% of the time for class 1, 20% for class 0
  mean imputation only          0.6294
  mean imputation + indicator   0.7969
  difference                   +0.1675
```

Sixteen and a half points, from a column of booleans and one keyword argument.

And the same feature can be leakage

The indicator is legitimate only if the reason for the hole will still exist when the model is deployed. If the test was skipped because the outcome was already known — a diagnosis recorded before the test was ordered — then you have encoded the answer into the features, and your test accuracy is measuring your data collection process rather than your model. Ask *when*, in the real timeline, the missingness is decided.

Try it yourself

Impute the eight marks with `SimpleImputer(strategy="most_frequent")`. All six observed values are distinct, so the mode is not unique: predict what scikit-learn does, then check (it takes the smallest, 55).

4 Handling outliers

4.1 Three kinds of outlier and one word for all of them

An outlier is an observation that deviates markedly from the others. That definition says nothing about what to do, because there are three completely different things it can be. **An error** — an age of 200, a blood pressure in the wrong units, a sensor stuck at its rail voltage: fix it or remove it. **A rare truth** — a genuinely enormous tumour or salary: keep it, and use a method that is not destroyed by it. **The point of the exercise** — fraud, a failing machine, a novel astronomical object: it is the signal, not the noise.

Key idea

No statistical rule can distinguish these three. A rule tells you a point is unusual; only knowledge of where the data came from tells you what it means. Detection is statistics; disposal is a judgement you must be able to defend in writing.

Iglewicz and Hoaglin, whose 1993 monograph is the standard reference and the source of the modified z-score below, separate three activities: *labelling* (flag candidates for investigation), *accommodation* (use a method robust enough not to care) and *identification* (formally test whether a point is an outlier). Most of what follows is labelling, which is what a preprocessing pipeline actually needs.

4.2 Three rules

$$\text{z-score:}\quad z_i = \frac{x_i - \bar{x}}{s}, \qquad \text{flag if } |z_i| > 3, \tag{3}$$

$$\text{Tukey fence:}\quad \mathrm{IQR} = Q_3 - Q_1, \qquad \text{flag if } x_i < Q_1 - 1.5\,\mathrm{IQR} \text{ or } x_i > Q_3 + 1.5\,\mathrm{IQR}, \tag{4}$$

$$\text{modified } z\text{:}\quad M_i = \frac{0.6745\,(x_i - \tilde{x})}{\mathrm{MAD}}, \qquad \text{flag if } |M_i| > 3.5. \tag{5}$$

Here $\tilde{x}$ is the median and $\mathrm{MAD} = \mathrm{median}_i\,|x_i - \tilde{x}|$ is the *median absolute deviation*. Three constants need explaining.

Why 0.6745: for Normal data $\mathrm{MAD} \approx 0.6745\,\sigma$, because 0.6745 is the 0.75 quantile of the standard Normal, so dividing by it puts M back on the z scale. **Why** 1.5: for Normal data $Q_3 + 1.5\,\mathrm{IQR}$ sits at about 2.70σ, so the fence flags 0.70% of observations; Tukey chose it as a convenient round number, not by derivation. **Why 3 and** 3.5: $|z| > 3$ flags 0.27% of Normal data, and Iglewicz and Hoaglin recommend 3.5 for the modified score.

The important column below is the last. The **breakdown point** of an estimator is the fraction of the sample you must corrupt before you can move the estimate arbitrarily far.

Rule	Breakdown point	Why
z-score	0%	one observation can move $\bar{x}$ and s as far as you like
Tukey / IQR	25%	the quartiles do not move until you corrupt a quarter of the data
modified z	50%	both the median and the MAD are medians

A breakdown point of 0% is not a technicality. It is the whole of the next subsection.

4.3 The rule that cannot see the outlier in front of it

Thirty-five readings from one sensor

Thirty genuine readings scattered around 10.0 with a spread of about 1, then the sensor sticks and reports exactly 100.0 five times. Nobody looking at this data could miss the problem.

The summary statistics of all thirty-five values:

mean $\bar{x}$	22.503	median $\tilde{x}$	9.950
sd s	32.109	MAD	0.540
Q_1	9.245	Q_3	10.330

Now score one stuck reading, $x = 100$, with each rule.

z-score. $z = (100 - 22.503)/32.109 = 2.4135$. The threshold is 3. **Not flagged.**
Tukey fence. IQR $= 10.330 - 9.245 = 1.085$, so the upper fence is $10.330 + 1.5(1.085) = 11.957$. Since $100 > 11.957$: **flagged**.
Modified z. $M = 0.6745(100 - 9.950)/0.540 = 112.48$ against a threshold of 3.5. **Flagged, by a factor of thirty-two.**

Why did the first rule fail? The five stuck readings dragged the mean from 9.586 up to 22.503 and inflated the standard deviation from 0.837 to 32.109. **They are being measured against a ruler that they themselves bent.** Remove them and recompute: the same reading of 100 would then score $z = (100 - 9.586)/0.837 = 108.1$.

```
z = (v - v.mean()) / v.std(ddof=1)
q1, q3 = np.percentile(v, [25, 75]); iqr = q3 - q1
mad = np.median(np.abs(v - np.median(v)))
mz = 0.6745 * (v - np.median(v)) / mad
```

```
35 readings: 30 genuine near 10.0, then 5 stuck at 100.0
  mean 22.503   sd 32.109   median 9.950   MAD 0.540
  Q1 9.245  Q3 10.330  IQR 1.085  upper fence Q3+1.5*IQR 11.957
  a stuck reading:  z = 2.4135  (needs > 3)   modified z = 112.48  (needs > 3.5)
rule                      flags   of which stuck   of which genuine
  |z| > 3                   0          0               0
  outside 1.5*IQR fence     6          5               1
  |modified z| > 3.5        5          5               0
sd WITHOUT the five stuck readings  0.837
z of a stuck reading would then be  108.1
```

The z-score rule found **none** of the five. The IQR fence found all five plus one genuine reading (a false positive, which is what a 0.70% Normal-data rate looks like on thirty points). The modified z found exactly the five.

Masking

This failure has a name. **Masking** is when the presence of one outlier prevents another from being detected, because the outliers between them inflate the very dispersion estimate they are being compared against. Its mirror image is **swamping**, where a genuine outlier drags the statistics far enough that innocent points get flagged too. NIST's handbook singles out masking as the reason you cannot apply a single-outlier test repeatedly and expect to find several.

Masking gets monotonically worse as contamination grows. Add stuck readings one at a time to the same thirty genuine ones and watch the largest $|z|$ in the sample:

```
    k= 0 stuck readings: largest |z|  2.518   largest mod. z     3.3
    k= 1 stuck readings: largest |z|  5.381   largest mod. z   119.3
```

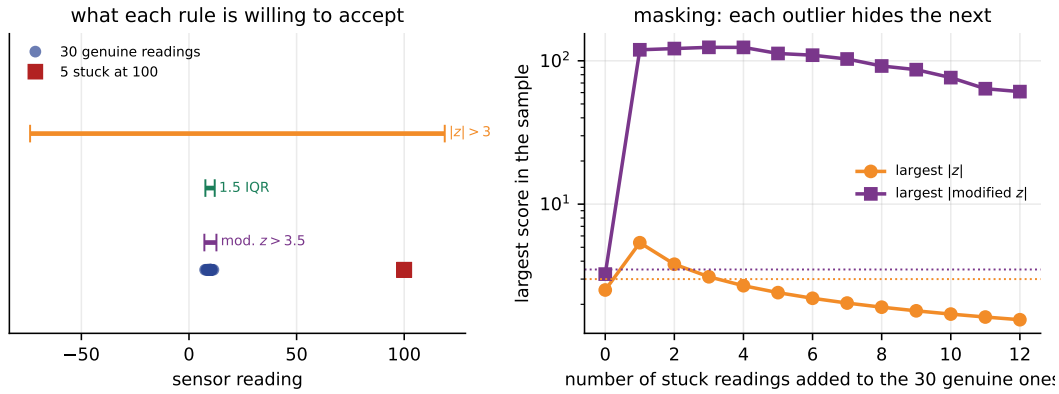

Figure 5: Left: the interval of values each rule is willing to accept. The z interval runs from -74 to $+119$ and comfortably contains the stuck readings at 100; the other two are slivers around 10. Right: the largest score in the sample as stuck readings are added, log scale, with the two thresholds dotted.

```
    k= 2 stuck readings: largest |z|  3.809   largest mod. z   121.7
    k= 3 stuck readings: largest |z|  3.113   largest mod. z   124.1
    k= 4 stuck readings: largest |z|  2.697   largest mod. z   124.1
    k= 5 stuck readings: largest |z|  2.414   largest mod. z   112.5
    k=10 stuck readings: largest |z|  1.710   largest mod. z    76.3
```

One outlier is caught easily at $z = 5.38$. Two are caught at 3.81, three barely at 3.11, and from the fourth onwards the rule never fires again — no matter how many more you add. The modified z stays above 60 throughout.

4.4 The rule also has a ceiling

There is a second, sharper way in which $|z| > 3$ fails, and it does not need any contamination at all.

The maximum attainable z-score

Take any sample of n values with sample standard deviation s (the $n-1$ one). Write $d_i = x_i - \bar{x}$, so $\sum_i d_i = 0$ and $\sum_i d_i^2 = (n-1)s^2$.
Suppose x_1 is the extreme point. The remaining $n-1$ deviations sum to $-d_1$, and among all vectors with a fixed sum the sum of squares is smallest when they are all equal, so

$$\sum_{i \geq 2} d_i^2 \;\geq\; (n-1)\left(\frac{-d_1}{n-1}\right)^2 = \frac{d_1^2}{n-1}.$$

Therefore

$$(n-1)s^2 = d_1^2 + \sum_{i \geq 2} d_i^2 \;\geq\; d_1^2\left(1 + \frac{1}{n-1}\right) = d_1^2\,\frac{n}{n-1},$$

which rearranges to

$$|z_1| = \frac{|d_1|}{s} \;\leq\; \frac{n-1}{\sqrt{n}}. \tag{6}$$

Equality holds exactly when the other $n-1$ values are all identical — the most extreme possible outlier.

```
n   largest possible |z|   can the rule |z| > 3 ever fire?
 3        1.1547              NO -- not ever
```

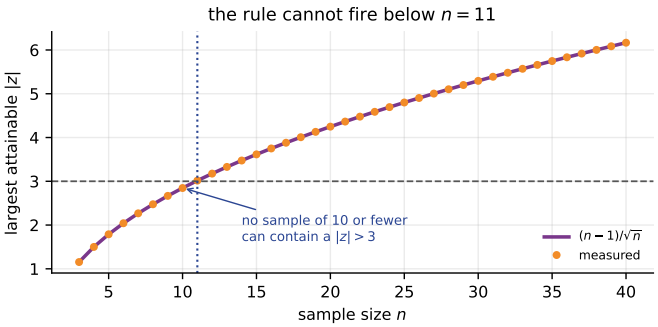

Figure 6: Equation (6), with the attained maxima measured directly. Theory and measurement agree to 9×10^{-16}. The crossing of the threshold happens between $n = 10$ and $n = 11$.

```
  5        1.7889              NO -- not ever
  8        2.4749              NO -- not ever
 10        2.8460              NO -- not ever
 11        3.0151              yes
 20        4.2485              yes
 50        6.9296              yes
closed form (n-1)/sqrt(n):  n=10 -> 2.8460,  n=11 -> 3.0151
```

Below $n = 11$ the rule is not strict, it is inert

$9/\sqrt{10} = 2.846 < 3$. If you have ten measurements and you write `if abs(z) > 3`, you have written a line of code whose value is `False` for every possible dataset. It is not conservative; it is broken. NIST states this bound in exactly these terms and recommends the modified z-score in the next sentence.

4.5 The opposite failure: skew is not contamination

So far the z-score has been too permissive. On the wrong kind of column all three rules are far too aggressive, and for a reason that has nothing to do with dirty data.

Take the most skewed column of `load_breast_cancer`: `'area error'`, skewness 5.43, kurtosis 48.8, and not one contaminated value in it.

```
most skewed column of breast_cancer: 'area error', skewness 5.433
  median 24.53   mean 40.34   sd 45.49   MAD 9.19
  rule                  flags     % of rows
  |z| > 3                 6         1.1%
  1.5*IQR fences         65        11.4%
  |modified z| > 3.5     81        14.2%
  largest reading 542.2:  z = 11.03,  modified z = 38.0
  under a perfect Normal these rules flag 0.27%, 0.70% and 0.35% of rows
```

Eighty-one “outliers” out of 569. Every one of these rules is derived under an assumption of symmetry: the z-score compares a two-sided deviation with a single s; the Tukey fence uses the same 1.5 IQR on both sides. On a long right tail they simply flag the tail.

The fix is not a different threshold. It is to make the column symmetric first.

```
  now Yeo-Johnson the column first (skewness 5.433 -> 0.069) and repeat
  |z| > 3                 0         0.0%
  1.5*IQR fences          1         0.2%
  |modified z| > 3.5      0         0.0%
```

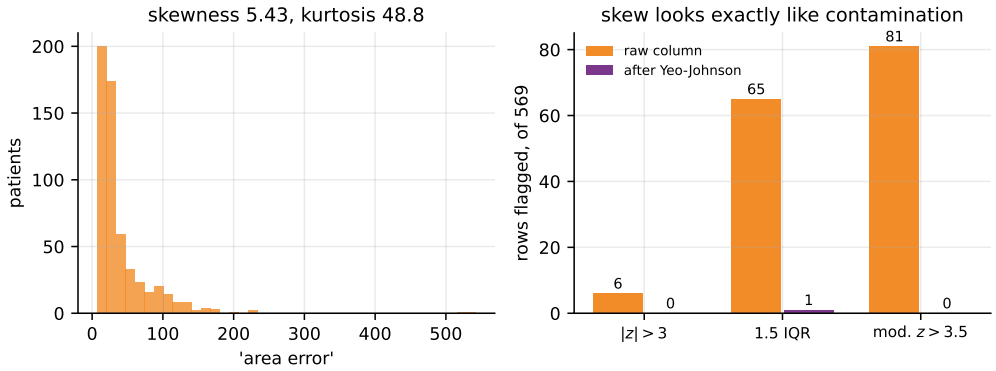

Figure 7: Left: the raw column. Right: how many of the 569 rows each rule condemns, before and after a single power transform. $6 \to 0$, $65 \to 1$, $81 \to 0$, and not one data value was deleted.

Key idea

Transform first, then look for outliers. An outlier rule applied to a skewed variable is mostly a skewness detector, and if you act on it you will delete the most informative 14% of your data for being large.

4.6 Numerical methods for handling outliers

Detection told you which points are unusual. Now: what do you do?

Binning and smoothing

The classical treatment of *noisy* numeric data — values wrong by a little rather than a lot — is to sort, bin, and replace each value by a summary of its bin.

Nine prices, three equal-depth bins

Sorted data: 4, 8, 15, 21, 21, 24, 25, 28, 34.
Equal-depth (equal-frequency) binning with depth 3 puts three values in each bin. *Equal-width* binning would instead cut the range $[4, 34]$ into three intervals of width 10, which here would give bins of size 2, 4 and 3.

Bin	Values	mean	median	boundaries
1	4, 8, 15	9	8	4 and 15
2	21, 21, 24	22	21	21 and 24
3	25, 28, 34	29	28	25 and 34

Smoothing by bin means replaces every value by its bin's mean: $9, 9, 9, 22, 22, 22, 29, 29, 29$.
Smoothing by bin medians gives $8, 8, 8, 21, 21, 21, 28, 28, 28$ — and unlike the means, these are values that actually occurred.
Smoothing by bin boundaries replaces each value by whichever *end* of its bin is closer. In bin 1, 8 is 4 away from 4 and 7 away from 15, so it becomes 4; 15 is already a boundary. The result is $4, 4, 15, 21, 21, 24, 25, 25, 34$, which preserves the range of each bin while flattening its interior.

```
sorted data  [4, 8, 15, 21, 21, 24, 25, 28, 34]
equal-frequency (equal-depth) bins of depth 3
  bin 1  [4, 8, 15]     mean  9.00   median  8   boundaries  4 and 15
  bin 2  [21, 21, 24]   mean 22.00   median 21   boundaries 21 and 24
```

```
  bin 3  [25, 28, 34]   mean 29.00   median 28   boundaries 25 and 34
smoothing by bin means       [9.0, 9.0, 9.0, 22.0, 22.0, 22.0, 29.0, 29.0, 29.0]
smoothing by bin medians     [8, 8, 8, 21, 21, 21, 28, 28, 28]
smoothing by bin boundaries  [4, 4, 15, 21, 21, 24, 25, 25, 34]
```

Binning is *local* smoothing: it replaces a value only by something from its own neighbourhood, so a single wild point contaminates one bin rather than the column. That is also its limitation — a bin of stuck readings smooths to a stuck value.

Trim, winsorise, cap, or leave alone

For a value that is wrong by a lot there are four choices. **Trim**: delete the row — loses data and, if the outliers are real, biases everything. **Winsorise**: clip everything below the α percentile up to it and everything above the $1-\alpha$ percentile down to it, keeping every row. **Cap at a fence**: the same, but the clipping point is $Q_3 + 1.5\,\mathrm{IQR}$, estimated from the data rather than chosen. **Leave it and use a robust method**: a median, a `RobustScaler`, a tree, an L_1 loss.

On the thirty-five sensor readings, where the truth is mean 9.586 and sd 0.837:

```
five ways to deal with the five stuck readings
  leave them alone          mean  22.503   sd  32.109   n 35
  winsorise at 5 and 95     mean  22.523   sd  32.100   n 35
  winsorise at 15 and 85    mean   9.903   sd   0.839   n 35
  cap at the IQR fence      mean   9.925   sd   1.143   n 35
  drop by modified z        mean   9.586   sd   0.837   n 30
  the 30 genuine readings   mean   9.586   sd   0.837   n 30
```

A fixed percentile is a guess about how much dirt there is

Winsorising at the 5th and 95th percentiles did **nothing at all**: mean 22.523 against 22.503. The contamination here is $5/35 = 14.3\%$, so clipping the outer 5% clips five stuck readings down to… other stuck readings. Winsorising at 15/85 works, and also clips five perfectly good readings. The IQR fence and the modified z need no such guess, because they estimate the location and spread from the middle of the data and let the tails fall where they will.

Robust scaling

The scaler you choose is an outlier decision whether you think of it that way or not.

```
StandardScaler().fit_transform(sensor.reshape(-1, 1)) # (x - mean) / sd
RobustScaler().fit_transform(sensor.reshape(-1, 1)) # (x - median) / IQR
```

```
StandardScaler against RobustScaler on the same 35 readings
  StandardScaler  30 genuine readings span -0.4747 to -0.3527  (width 0.1220)
  RobustScaler    30 genuine readings span -2.2765 to  1.2811  (width 3.5576)
```

`StandardScaler` spent its entire dynamic range accommodating five wrong numbers: the thirty good readings now occupy 0.122 of a unit, and to any distance-based model they are effectively identical. `RobustScaler` centres on the median and divides by the IQR, so the thirty good readings spread over 3.56 units and the outliers simply sit far away, which is where they belong.

Try it yourself

Run the three rules on `load_iris` petal length: you should get zero flags from all three. Change one value to 100 and re-run — all three fire, because the single-outlier case is the one the z-score handles fine. Now add a second and a third 100 and watch the z-score give up.

5 Data transformation

5.1 Why transform a variable

Four honest reasons. **Symmetry**: a mean and a standard deviation describe a symmetric distribution well and a skewed one badly. **Linearity**: a relationship curved in x is often straight in $\log x$. **Constant spread**: if residual variance grows with the fitted value, a transformation often flattens it. And **so that the outlier rules work** — Section 4.5.

The bad reason is "because everybody does". A transformation changes what a coefficient means, and if you cannot state the new meaning you should not apply it.

Han, Kamber and Pei list smoothing, attribute construction, aggregation, normalisation, discretisation and concept-hierarchy generation under the same heading of "data transformation". Smoothing was Section 4.6; normalisation, discretisation and attribute construction belong with feature engineering in the next lecture. This section is about the one that changes the *shape* of a distribution.

5.2 The ladder of powers, by hand

Arrange the simple transformations by the power they raise x to:

$$\ldots, \quad -\frac{1}{x}, \quad -\frac{1}{\sqrt{x}}, \quad \log x, \quad \sqrt{x}, \quad x, \quad x^2, \quad x^3, \quad \ldots$$

Moving **down** the ladder pulls in a long right tail; moving **up** pulls in a long left tail. The further you move, the stronger the effect.

Why log is the useful middle rung

Take four values spanning two orders of magnitude: 1, 4, 16, 64. They are in geometric progression with ratio 4, so the gaps grow: 3, 12, 48.
Apply log: 0, 1.3863, 2.7726, 4.1589. The gaps are now 1.3863, 1.3863, 1.3863 — exactly equal, because $\log(4x) = \log x + \log 4$.
The log turns multiplication into addition. A variable whose values are naturally *ratios* — incomes, populations, areas, concentrations, prices — has a skewed distribution on the original scale and a symmetric one on the log scale, and that is not a coincidence: it is what "multiplicative" means. Reach for log first whenever "twice as big" is a more natural statement about your variable than "ten units bigger".

5.3 Four transformations of one real column

`'area error'` again: 569 values, minimum 6.80, maximum 542.2, skewness 5.43.

```
for name, v in (("none", col), ("sqrt", np.sqrt(col)),
                ("log", np.log(col)), ("1/sqrt", 1/np.sqrt(col))):
    print(name, stats.skew(v), stats.kurtosis(v))
```

```
column 'area error',  n = 569,  min = 6.8020, max = 542.2
  transformation        skewness    kurtosis
  none                    5.4328     48.7672
  sqrt(x)                 2.1363      7.8193
  log(x)                  0.7955      0.3622
  1/sqrt(x)               0.0447     -0.3685
  Box-Cox, lam=-0.436     0.0584     -0.4094
  Yeo-Johnson             0.0691     -0.4561
```

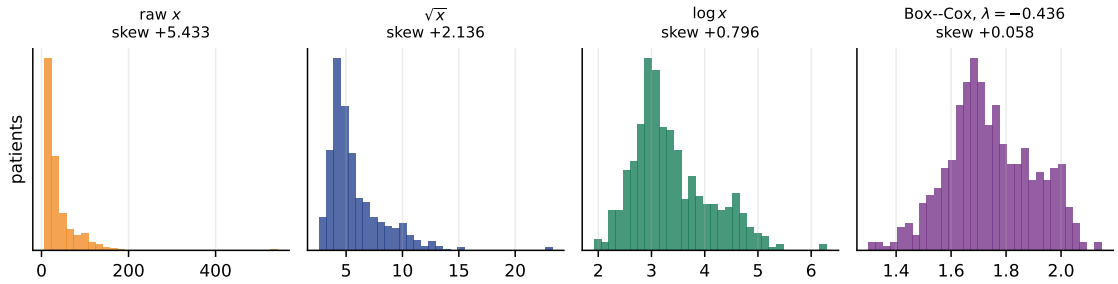

Figure 8: The same 569 numbers on four different rulers. No value has been deleted and no value has changed its rank. Only the fourth panel is a variable you can sensibly summarise with a mean and a standard deviation.

Read down the skewness column: $5.43 \to 2.14 \to 0.80 \to 0.04$. The square root was not nearly enough, the log almost did it, and the reciprocal square root went slightly past zero.

The ladder is discrete, and the right rung for this column is somewhere between log and $1/\sqrt{x}$. That gap is exactly what the next idea fills.

5.4 Box–Cox and Yeo–Johnson

Box and Cox (1964) embedded the whole ladder in one continuous family:

$$x^{(\lambda)} = \begin{cases} \dfrac{x^{\lambda} - 1}{\lambda}, & \lambda \neq 0, \\ \log x, & \lambda = 0, \end{cases} \qquad x > 0. \tag{7}$$

Two details of Equation (7) are worth a moment.

Why the -1 and the division by λ. They make the family continuous at $\lambda = 0$: without them $x^{\lambda} \to 1$ and the transformation would collapse to a constant, while with them l'Hôpital's rule gives $(x^{\lambda} - 1)/\lambda \to \log x$. Numerically:

```
    check lambda->0:  (x**0.001 - 1)/0.001 = [0.0, 1.3873, 2.7764, 4.1675]
    log(x)                                 = [0.0, 1.3863, 2.7726, 4.1589]
```

Why $x > 0$. x^{λ} is not real for negative x and fractional λ. Yeo and Johnson (2000) patched this by applying the Box–Cox form to $x+1$ on the positive side and a mirrored version to $-x+1$ on the negative side, giving a family that is defined on the whole real line and still smooth at zero. That is what scikit-learn's `PowerTransformer` uses by default.

Every named rung is a value of λ:

```
Box-Cox is one family; lambda picks the member
  lambda -1.00   1/x  (reciprocal)  skewness  -0.8635
  lambda -0.50   1/sqrt(x)          skewness  -0.0447
  lambda +0.00   log(x)             skewness  +0.7955
  lambda +0.50   sqrt(x)            skewness  +2.1363
  lambda +1.00   x  (no change)     skewness  +5.4328
  maximum-likelihood lambda from scipy   -0.4357
  skewness there                         +0.0584
```

λ is chosen by likelihood, not by skewness

`scipy.stats.boxcox` picks the λ that maximises the Gaussian log-likelihood of the transformed data, not the one that zeroes the skewness. On this column those are -0.4357 and

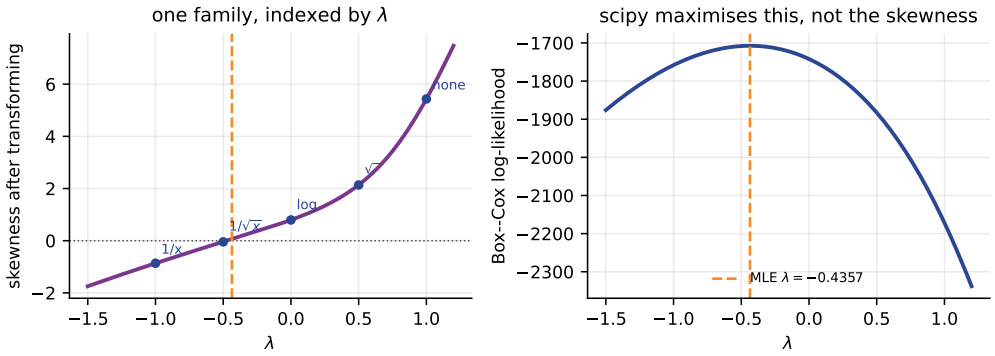

Figure 9: Left: skewness as a function of λ, with the five named rungs of the ladder marked. Right: the Box–Cox log-likelihood, whose maximum at $\lambda = -0.4357$ is what scipy returns.

-0.4688 — close, but they are answers to different questions, and on a heavy-tailed column they can differ a lot. If what you need is symmetry, check the skewness afterwards rather than assuming.

```
a column that goes negative cannot use Box-Cox
  diabetes 'bp': min -0.1124  max 0.1320  skewness +0.2897
  stats.boxcox(bp) -> ValueError: Data must be positive.
  Yeo-Johnson handles it: skewness +0.2897 -> +0.0226
```

5.5 Does it help the model?

Sometimes, and which model it is matters more than how skewed the column was.

```
does removing the skew help a model? 20 random splits
  GaussianNB           scale only 0.9304   Yeo-Johnson first 0.9462   +0.0157  (16/20)
  LogisticRegression   scale only 0.9762   Yeo-Johnson first 0.9766   +0.0003  (5/20)
```

Gaussian naive Bayes gains 1.6 points on 16 of 20 splits, because it *assumes* each feature is Normal within each class and the transform makes that assumption closer to true. Logistic regression gains 0.0003 and wins 5 splits of 20 — noise — because it assumes nothing about the distribution of the features.

Key idea

Transform because your model assumes symmetry, because you need an outlier rule to work, or because you need to read the variable. Not as a ritual. And when you do transform, remember that a coefficient on $\log x$ is a statement about percentage change, and say so in the write-up.

Try it yourself

Take `load_breast_cancer`, compute `stats.skew(Xb, axis=0)` and find how many of the thirty columns have skewness above 1. Then apply `PowerTransformer()` to the whole matrix and count again. (You should find 12 before and 0 after.)

6 The order of operations

6.1 The rule

Everything in this lecture learns a number from the data: imputation learns a column mean, median or mode, or a whole fitted model; outlier handling learns a threshold, a fence, or *which rows to keep*; transformation learns λ; scaling learns a mean and a standard deviation, or a median and an IQR.

Key idea

Every number used to clean the data must be computed from the **training rows only**, and then applied unchanged to the test rows. Learn it on train, apply it everywhere: `fit` on train, `transform` on both.

Break that and your test score stops being an estimate of performance on unseen data, because the data was not unseen. Every course tells you that. What courses rarely tell you is *how much* it costs — and the answer spans three orders of magnitude depending on which step you got wrong. So we measured three.

6.2 Leak A: fit the scaler on everything

The classic. `StandardScaler().fit(X)` before the split, so the mean and standard deviation are computed from the test rows too.

```
sc = StandardScaler().fit(Xb) # <-- sees the test set
m = LogisticRegression(max_iter=5000).fit(sc.transform(Xtr), ytr)
leaky = m.score(sc.transform(Xte), yte)
honest = make_pipeline(StandardScaler(),
                       LogisticRegression(max_iter=5000)
                       ).fit(Xtr, ytr).score(Xte, yte)
```

```
LEAK A -- fit the scaler on everything, then split.  300 splits each
  n_train = 20  leaky 0.9338  honest 0.9304  gap +0.0034  t  +6.0  win 172/300
  n_train = 60  leaky 0.9587  honest 0.9577  gap +0.0011  t  +4.3  win 146/300
  n_train = 427 leaky 0.9774  honest 0.9772  gap +0.0002  t  +1.7  win 18/300
```

The leak is **real** — at $n_{\text{train}} = 20$, $t = +6.0$ over 300 splits is not noise — and **tiny**: three tenths of a percentage point, shrinking to nothing as the training set grows, because the mean of 427 rows and the mean of 569 rows are almost the same number.

6.3 Leak B: fit the imputer on everything

```
LEAK B -- fit the imputer on everything.  30% missing, 60 splits
  mean          leaky 0.9586  honest 0.9583  gap +0.0003  t  +0.8  win 9/60
  KNN k=5       leaky 0.9535  honest 0.9552  gap -0.0017  t  -1.4  win 18/60
```

Report what you measured, not what you expected

We expected KNN imputation fitted on the whole dataset to leak visibly, because it literally copies values between rows. It did not. Over sixty splits at 30% missingness the gap was -0.0017 with $t = -1.4$: if anything the leaky version scored *lower*, and neither result is distinguishable from zero.

That does not make the practice acceptable. A leaky estimate is not interpretable even when it happens not to be optimistic — you no longer know what it is an estimate of. But it does mean you should not go into an examination claiming that imputing before the split

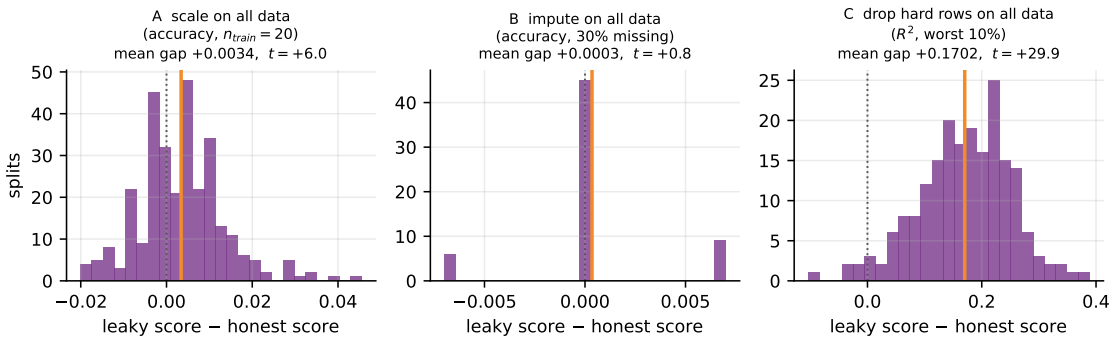

Figure 10: Per-split difference, leaky minus honest, for the three sins. Note the horizontal scales: 0.04, 0.008 and 0.4. The orange line is the mean, the dotted line is zero.

produces a large optimistic bias, because on this data it produced none.

6.4 Leak C: clean the rows using the model, then split

Now the one that does real damage, and the one that is most human. You fit a model, look at the rows it gets badly wrong, decide they must be outliers, delete them, and start again.

```
m = LinearRegression().fit(Xd, yd) # <-- fitted on all 442 rows
res = np.abs(yd - m.predict(Xd))
keep = res <= np.quantile(res, 0.90) # drop the worst 10%
for seed in range(0, 200): # a sweep: 200 different splits
    A, B, a, b = train_test_split(Xd[keep], yd[keep], test_size=0.25,
                                  random_state=seed)
    leaky = LinearRegression().fit(A, a).score(B, b)
```

```
LEAK C -- drop the rows your model cannot fit, THEN split.  200 splits
  drop worst 5% leaky 0.5850  honest 0.4730  gap +0.1120  t +19.2  win 181/200
  drop worst 10% leaky 0.6390  honest 0.4688  gap +0.1702  t +29.9  win 192/200
  drop worst 20% leaky 0.7022  honest 0.4732  gap +0.2291  t +45.0  win 200/200
```

The honest column is the control: split first, fit on the training rows only, drop the worst-fitted *training* rows, evaluate on the untouched test set. It barely moves — 0.4730, 0.4688, 0.4732. The leaky column climbs from 0.4688 to 0.7022, and nothing about the model improved: the filter simply removed from the *test* set precisely the patients the model was going to fail on.

Key idea

The size of a leak is the amount of test information the step absorbs. A column mean absorbs almost nothing — worth 0.0034. A decision about *which rows survive* absorbs a great deal — worth 0.17 of R^2, on 192 splits out of 200. Learn to ask what the step learned from the test set, rather than memorising a list of forbidden operations.

6.5 The pipeline that cannot leak

Every fix in this section is the same fix.

```
from sklearn.pipeline import Pipeline
pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median", add_indicator=True)),
    ("power", PowerTransformer(method="yeo-johnson")),
    ("scale", StandardScaler()),
```

```
    ("model", LogisticRegression(max_iter=5000)),
])
print(cross_val_score(pipe, X_with_holes, y, cv=5).mean())
```

```
one Pipeline, 5-fold cross-validation, 10% of cells missing
  steps: impute -> power -> scale -> model
  cross_val_score  0.9578  +/- 0.0140
every step is refitted inside every fold, so no fold ever sees its own
test rows -- and there is nothing left for you to get wrong by hand
```

`cross_val_score` calls `fit` on the pipeline five times, once per fold. Each time the medians, the λs, the means and the standard deviations are recomputed from four fifths of the data, and the held-out fifth is only ever passed to `transform`. You cannot get the order wrong, because you no longer write the order.

Row-dropping does not fit in a Pipeline, and that is the point

A scikit-learn transformer may change the columns of X but not its rows, so there is no `OutlierRemover` step you can slot in. That is a design decision, not an omission: dropping rows during `transform` would break the correspondence with y, and dropping them from the test fold is exactly Leak C. If you must remove outliers, do it to the training set, by hand, after the split — and evaluate on everything.

Try it yourself

Reproduce Leak C for yourself with ten random seeds. Then change the honest version so that it also drops the worst-fitted rows from the *test* set, and watch it become the leaky version. Being able to write the bug deliberately is the best evidence that you will not write it accidentally.

7 Summary

1. Listwise deletion survives with probability $(1-p)^d$, exponential in the number of columns. Five per cent of cells missing across thirty features left 114 patients of 569.

2. Mean imputation leaves the mean alone and multiplies the variance by exactly m/n: measured 0.9486 against $\sqrt{0.9} = 0.9487$ and 0.5447 against $\sqrt{0.3} = 0.5477$. The shrinkage propagates — corr(BMI, y) fell from 0.5865 to 0.4538 at 40% imputed.

3. MCAR / MAR / MNAR is the only taxonomy that changes what you can do. Complete-case bias was -0.10, $+1.35$ and -2.95 kg/m^2; MICE fixed the MAR case ($+1.35 \to -0.16$) and could not fix MNAR ($-2.95 \to -2.58$).

4. Under MNAR, mean imputation flipped the sign of a regression coefficient: $+5.603$ became -0.413.

5. Dropping incomplete rows cost 0.05 of R^2 at 10% missingness and 5.17 at 30%. The choice among five imputers cost 0.003. **Spend your attention accordingly.**

6. `add_indicator=True` was worth $+0.1675$ of accuracy when the missingness itself carried the signal.

7. **Thirty good sensor readings plus five stuck at 100: the $|z| > 3$ rule flagged none of them** ($z = 2.4135$). The IQR fence flagged five plus one false positive; the modified z flagged exactly five, at $M = 112.5$.

8. Masking is monotone: one outlier is caught at $z = 5.38$, three barely at 3.11, and from four onwards the rule never fires again. And no sample can contain a $|z|$ larger than $(n-1)/\sqrt{n}$, so for $n \leq 10$ the rule $|z| > 3$ is not strict but inert.

9. On a column with skewness 5.43 and no contamination, the three rules flagged 6, 65 and 81 rows of 569. After one Yeo–Johnson: 0, 1 and 0. **Transform first, then look for outliers.**

10. Box–Cox found $\lambda = -0.4357$ by maximum likelihood and took the skewness from 5.4328 to 0.0584. Every named rung of the ladder is a value of λ; Yeo–Johnson is the version that survives negative values.

11. Removing the skew was worth +0.0157 to Gaussian naive Bayes, which assumes normality, and +0.0003 to logistic regression, which does not.

12. Leaking the scaler was worth +0.0034 ($t = 6.0$); leaking the imputer was worth nothing measurable; leaking a row-dropping decision was worth +0.1702 of R^2 ($t = 29.9$). Use a `Pipeline` and none of them can happen.

8 Practice questions

Attempt these before reading Section 9. Questions 1, 4, 5, 6 and 7 are pure hand arithmetic and are the ones most likely to appear on a paper.

1. Five lab scores out of 25 are recorded as 18, 22, ?, 15, ?. (a) Compute the mean, the median and the sample standard deviation of what was observed. (b) Impute both holes with the mean, then with the median, and recompute the mean and standard deviation in each case. (c) Check your standard deviation against the formula of Section 3.3.

2. A household survey asks for annual income. Thirty per cent of respondents leave it blank, and follow-up shows the blanks are concentrated among the highest earners. (a) Name the mechanism. (b) What does complete-case analysis do to the estimated mean income — which direction, and can you bound it? (c) The survey also records occupation and postcode, both complete. Under what circumstance would `IterativeImputer` fix the problem, and why does it not fix it here?

3. A table has d columns and each cell is independently missing with probability p. (a) Write down the probability a row survives listwise deletion. (b) At $p = 0.05$, how many columns before you expect to lose half the rows? (c) You have 50 columns and $p = 0.10$; you need 1000 complete rows. How many rows must you collect?

4. Twelve measurements, already sorted:

   $$41,\ 44,\ 45,\ 47,\ 48,\ 49,\ 51,\ 52,\ 54,\ 57,\ 120,\ 125.$$

   Compute the mean, sample standard deviation, median, MAD, Q_1, Q_3 and the Tukey fences. Then score 120 and 125 with all three rules of Section 4.2 and say which rule flags what. Explain the result.

5. Prove that $|z_i| \leq (n-1)/\sqrt{n}$ for any sample of size n using the $n-1$ standard deviation. What is the smallest n for which a rule of $|z| > 3$ can ever fire? What about $|z| > 2$?

6. Smooth the twelve values 8, 9, 15, 16, 16, 20, 21, 21, 24, 26, 27, 30 using equal-depth bins of depth 3, by bin means, by bin medians and by bin boundaries. Which of the three never invents a value that did not occur in the data?

7. Apply the Box–Cox transformation of Equation (7) to $x = 1,\ 4,\ 16,\ 64$ for $\lambda = 1,\ 0.5,\ 0,\ -1$. (a) Why is the first transformed value zero in every case? (b) Show numerically that the $\lambda \neq 0$ formula tends to $\log x$ as $\lambda \to 0$. (c) Why does the same formula fail on the diabetes `bp` column, and what do you use instead?

8. On the thirty-five sensor readings of Section 4.3, winsorising at the 5th and 95th percentiles changed the mean from 22.503 to 22.523 — effectively nothing. Explain why, and state the general rule for choosing a winsorising percentile. What would you use instead if you did not know the contamination rate?

9. A colleague reports $R^2 = 0.64$ on the diabetes data with an ordinary linear regression. You reproduce their code and get 0.47. Their notebook, in order, does: (i) load the data; (ii) fit a linear model to all 442 rows and delete the 10% with the largest residual; (iii) `StandardScaler().fit_transform(X)` on what is left; (iv) `train_test_split`; (v) fit and report. Identify every problem, say which one accounts for essentially all of the gap, and give the corrected code.

10. You are handed a table of 8000 patient records with 40 numeric columns. Twelve per cent of the cells are missing; three columns have skewness above 4; one column is a laboratory value that is recorded only when a clinician orders the test. Design the cleaning pipeline. For each decision, cite a measured number from this lecture that supports it.

9 Solutions

1. Observed: 18, 22, 15, so $m = 3$ and $n = 5$.

(a) Mean $= 55/3 = 18.3333$ and median $= 18$. For the sample standard deviation, the deviations are -0.3333, 3.6667 and -3.3333; their squares sum to $0.1111 + 13.4444 + 11.1111 = 24.6667$; dividing by $m - 1 = 2$ gives 12.3333, so $s = 3.5119$.

(b) Mean-imputed: 18, 22, 18.3333, 15, 18.3333. The mean is unchanged at 18.3333; the standard deviation is 2.4833. Median-imputed: 18, 22, 18, 15, 18, whose mean has *moved* to 18.2 and whose standard deviation is 2.4900.

(c) $\sqrt{(m-1)/(n-1)} = \sqrt{2/4} = 0.7071$, and $2.4833/3.5119 = 0.7071$. Exact.

```
Q1  observed [18.0, 22.0, 15.0]  m=3  n=5
    mean 18.3333   median 18.0000   sd(ddof=1) 3.5119
    mean   imputed: mean 18.3333  sd 2.4833  ratio 0.7071
    median imputed: mean 18.2000  sd 2.4900  ratio 0.7090
    sqrt((m-1)/(n-1)) = sqrt(2/4) = 0.7071
```

Note that the median-imputed ratio, 0.7090, is *not* exactly $\sqrt{0.5}$. The derivation in Section 3.3 used the fact that the filled value contributes zero deviation, which is only true for the mean.

2. (a) **MNAR**: the probability of a value being missing depends on the value itself.

(b) It **underestimates** mean income, because the discarded observations are systematically the large ones. You cannot bound the bias from the data — that is what MNAR means — but you can bound it from outside: a known population maximum or a tax authority's aggregate gives a worst case. A sensitivity analysis over plausible assumptions is the standard practice.

(c) `IterativeImputer` learns $P(\text{income} \mid \text{occupation}, \text{postcode})$ *from the respondents who answered*. If, within any given occupation and postcode, the answerers and the refusers had the same income distribution, the mechanism would be MAR and the imputer would recover the truth. Here a high earner refuses *because* they are a high earner, so within every occupation

and postcode the observed incomes are the low ones and the imputer faithfully reproduces the bias. Measured: MAR bias $+1.3473 \to -0.1588$ under MICE, MNAR bias $-2.9505 \to -2.5834$.

3. (a) $P = (1-p)^d$.

(b) $(0.95)^d = 0.5$ gives $d = \log 0.5 / \log 0.95 = 13.5$, so from about fourteen columns onwards you are discarding more than half your data.

(c) $(0.90)^{50} = 0.00515$, so you need $1000/0.00515 = 194{,}000$ rows to end up with 1000 complete ones. If that sounds absurd, it is — and it is the argument for imputation in one line.

```
Q3  rows surviving listwise deletion, (1-p)**d
    d= 5   p=0.01 0.951   p=0.05 0.774   p=0.10 0.590   half-life d at p=0.05: 13.5
    d=10   p=0.01 0.904   p=0.05 0.599   p=0.10 0.349   half-life d at p=0.05: 13.5
    d=20   p=0.01 0.818   p=0.05 0.358   p=0.10 0.122   half-life d at p=0.05: 13.5
    d=30   p=0.01 0.740   p=0.05 0.215   p=0.10 0.042   half-life d at p=0.05: 13.5
    d=50   p=0.01 0.605   p=0.05 0.077   p=0.10 0.005   half-life d at p=0.05: 13.5
```

4. The mean is $733/12 = 61.0833$ and the sample standard deviation is 29.0406 — both dragged upwards by the last two values. The median is $(49+51)/2 = 50$. Absolute deviations from 50 are $9, 6, 5, 3, 2, 1, 1, 2, 4, 7, 70, 75$; their median is 4.5, so MAD = 4.50. The quartiles are $Q_1 = 46.50$ and $Q_3 = 54.75$, so IQR = 8.25 and the fences are 34.125 and 67.125.

```
Q4  n=12  mean 61.0833  sd 29.0406  median 50.00  MAD 4.50
    Q1 46.50  Q3 54.75  IQR 8.25  fences 34.125 and 67.125
    z of 120 = 2.0288   z of 125 = 2.2009   ceiling (n-1)/sqrt(n) = 3.1754
    modified z of 120 = 10.49   of 125 = 11.24
    flags:  |z|>3 0      IQR 2      |mod z|>3.5 2
    with the two large values removed: sd 4.8488, z of 120 would be 14.7
```

The z-scores are 2.03 and 2.20, both comfortably inside the threshold, so $|z| > 3$ flags **nothing**. Both other rules flag both points. Two explanations, and you should give both.

Masking. The two large values inflated s from 4.85 (on the ten genuine measurements) to 29.04, a factor of six; divided by the uncontaminated s, 120 would score $z = 14.7$. *The ceiling.* With $n = 12$ the largest attainable $|z|$ is $11/\sqrt{12} = 3.1754$, so even a value of 10^9 would have scored only 3.18 — the rule had almost no room to work in even before the masking.

5. Write $d_i = x_i - \bar{x}$, so $\sum_i d_i = 0$ and $\sum_i d_i^2 = (n-1)s^2$. Fix attention on d_1. The other $n-1$ deviations sum to $-d_1$, and for a fixed sum the sum of squares is minimised when they are all equal, so $\sum_{i \geq 2} d_i^2 \geq d_1^2/(n-1)$. Hence

$$(n-1)s^2 \;\geq\; d_1^2 + \frac{d_1^2}{n-1} = d_1^2 \cdot \frac{n}{n-1}, \qquad \text{so} \qquad \frac{d_1^2}{s^2} \leq \frac{(n-1)^2}{n}, \qquad |z_1| \leq \frac{n-1}{\sqrt{n}}.$$

For $|z| > 3$ we need $(n-1)/\sqrt{n} > 3$, i.e. $n - 1 > 3\sqrt{n}$. At $n = 10$, $9/\sqrt{10} = 2.846$; at $n = 11$, $10/\sqrt{11} = 3.015$. So $n \geq 11$. For $|z| > 2$: at $n = 5$, $4/\sqrt{5} = 1.789$; at $n = 6$, $5/\sqrt{6} = 2.041$. So $n \geq 6$.

6. Depth 3 gives four bins.

```
Q6  sorted [8, 9, 15, 16, 16, 20, 21, 21, 24, 26, 27, 30]
    bin 1 [8, 9, 15]     mean  10.67  median  9  boundaries  8,15
    bin 2 [16, 16, 20]   mean  17.33  median 16  boundaries 16,20
    bin 3 [21, 21, 24]   mean  22.00  median 21  boundaries 21,24
    bin 4 [26, 27, 30]   mean  27.67  median 27  boundaries 26,30
    by means      [10.67, 10.67, 10.67, 17.33, 17.33, 17.33, 22.0, 22.0, 22.0, 27.67, 27.67,
    27.67]
    by medians    [9, 9, 9, 16, 16, 16, 21, 21, 21, 27, 27, 27]
    by boundaries [8, 8, 15, 16, 16, 20, 21, 21, 24, 26, 26, 30]
```

Check bin 1 by hand: $8, 9, 15$ has mean $32/3 = 10.67$ and median 9; for the boundaries, 9 is 1 from 8 and 6 from 15, so it becomes 8. **Bin medians and bin boundaries never invent a value**; bin means can produce 10.67 when every measurement was an integer, which matters if the column is a count or a category code.

7. (a) $x = 1$ gives $(1^\lambda - 1)/\lambda = 0$ for every $\lambda \neq 0$, and $\log 1 = 0$. The -1 in the numerator is what fixes the value 1 to map to 0 throughout the family.

```
Q7  x = [1.0, 4.0, 16.0, 64.0]
    lambda +1.0 -> [0.0, 3.0, 15.0, 63.0]
    lambda +0.5 -> [0.0, 2.0, 6.0, 14.0]
    lambda +0.0 -> [0.0, 1.3863, 2.7726, 4.1589]
    lambda -1.0 -> [0.0, 0.75, 0.9375, 0.9844]
```

Check $\lambda = 0.5$ by hand: $(\sqrt{4} - 1)/0.5 = (2 - 1)/0.5 = 2$; $(\sqrt{16} - 1)/0.5 = 6$; $(\sqrt{64} - 1)/0.5 = (8 - 1)/0.5 = 14$. Notice how the spacing changes: at $\lambda = 1$ the gaps are $3, 12, 48$; at $\lambda = 0$ they are equal. That is the ladder working.

(b) Put $\lambda = 0.001$:

```
    check lambda->0:  (x**0.001 - 1)/0.001 = [0.0, 1.3873, 2.7764, 4.1675]
    log(x)                                 = [0.0, 1.3863, 2.7726, 4.1589]
```

Agreement to three decimals, improving as $\lambda \to 0$: analytically $x^\lambda = e^{\lambda \log x} = 1 + \lambda \log x + O(\lambda^2)$.

(c) The diabetes `bp` column runs from -0.1124 to $+0.1320$ and x^λ is not real for negative x at fractional λ; scipy raises `ValueError: Data must be positive`. Use Yeo–Johnson, which took the skewness from $+0.2897$ to $+0.0226$. Shifting the column by a constant and then using Box–Cox is a common alternative, but the answer then depends on the shift, which is one more thing to justify.

8. Winsorising at $\alpha = 5\%$ clips the outer 5% of the *sample*, but the contamination here is $5/35 = 14.3\%$. Clipping the top 5% means clipping the top 1.75 observations — all of them stuck readings at 100 — down to the 95th percentile, which is *also* a stuck reading at 100. Nothing moves.

```
Q8  winsorising the 35 sensor readings at symmetric percentiles
     2/98  mean  22.515  sd  32.104   (truth 9.586 / 0.837)
     5/95  mean  22.523  sd  32.100   (truth 9.586 / 0.837)
    10/90  mean  22.569  sd  32.079   (truth 9.586 / 0.837)
    15/85  mean   9.903  sd   0.839   (truth 9.586 / 0.837)
    20/80  mean   9.824  sd   0.565   (truth 9.586 / 0.837)
    contamination is 5/35 = 14.3%
```

The general rule: α **must exceed the contamination rate**, which you rarely know, so you are guessing. Overshoot and you clip good data — at 20/80 the standard deviation falls to 0.565 against a true 0.837, because sixteen genuine readings have been clipped. If you do not know the rate, use a rule that estimates the centre and spread from the middle of the data rather than from a fixed tail quantile: the 1.5 IQR fence gave mean 9.925, and the modified z-score recovered the thirty genuine readings exactly.

9. There are three problems, and they are not equally serious.

Problem 1, and it is the whole gap. Step (ii) fits a model to all 442 rows and deletes the ones it fits worst, *before* the split. Those rows are then absent from the test set, so the test set no longer contains the patients the model would fail on. Measured over 200 splits, this alone moves R^2 from 0.4688 to 0.6390: a gap of $+0.1702$ with $t = 29.9$, and the leaky number wins on 192 of the 200. It accounts for essentially all of the 0.64 against 0.47.

Problem 2, real but small. Step (iii) fits the scaler on all surviving rows, so the test rows contribute to the mean and standard deviation. Measured separately: +0.0034 at a training set of 20 rows, +0.0002 at 427 — detectable, practically irrelevant here, and free to fix.

Problem 3, not leakage but still wrong. Deleting the worst-fitted 10% defines "outlier" as "inconvenient for my model", which guarantees the survivors agree with the model. Even done correctly, inside the training set only, it buys nothing: 0.4730, 0.4688, 0.4732 for 5%, 10% and 20%.

Corrected:

```
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25,
                                          random_state=0)
model = make_pipeline(StandardScaler(), LinearRegression())
model.fit(X_tr, y_tr) # scaler fitted on train only
print(model.score(X_te, y_te)) # evaluated on every test row
```

If outlier removal is genuinely wanted, do it to `X_tr` and `y_tr` after the split, with a criterion that does not consult the model — the modified z-score, say — and never touch `X_te`.

10. A defensible pipeline, with the evidence for each step.

Look before you clean. Count missing cells per column and per row and plot every histogram. At 12% per cell across 40 columns, listwise deletion keeps $(0.88)^{40} = 0.6\%$ of the rows — 48 patients of 8000. That one line of arithmetic settles whether to delete.

Missing values. Impute, with the median, because three columns are strongly skewed. Which imputer barely matters: our measured spread across five was 0.003 of R^2 at 10% missingness against 0.05 for deletion. Consider `KNNImputer` if 12% behaves like our 30% case, where KNN gave 0.4591 against mean imputation's 0.4351 — but watch the variance, since MICE there gave 0.2639 ± 0.5004.

The clinician-ordered laboratory value. Set `add_indicator=True`: when missingness carried signal it was worth +0.1675 of accuracy (Section 3.6). Then ask whether the test is ordered *because* of something already known about the outcome — if so the indicator is leakage and must go. Suspect MNAR here and say so: our MNAR case flipped a coefficient from +5.603 to −0.413.

Skew, before outliers. Apply `PowerTransformer` (Yeo–Johnson, so negative values are safe) to the three skewed columns. On a column with skewness 5.43 this cut the rows flagged as outliers from 6, 65 and 81 to 0, 1 and 0; running an outlier rule first would have deleted the top 14% of a legitimate distribution.

Outliers. Use the modified z-score, not $|z| > 3$: on our sensor data the z rule found zero of five obvious outliers and the modified z found exactly five. Investigate rather than delete. If a flagged value is impossible, drop that row from the training set only; if it is merely large, keep it and use `RobustScaler`, which kept the thirty genuine readings spread over 3.56 units where `StandardScaler` squeezed them into 0.122.

Order. Everything except the row deletion goes into one `Pipeline`; the row deletion happens by hand after the split, on the training set only. Measured cost of getting this wrong: +0.0034 for a leaked scaler, +0.1702 for a leaked row-dropping decision.

10 References and further reading

All links were checked on 22 August 2026. Free resources are marked [**free**].

Textbooks

- Han, Kamber and Pei, *Data Mining: Concepts and Techniques*, 3rd edition, Morgan Kaufmann, 2012. **The prescribed text.** §3.1 the four preprocessing tasks; §3.2 data cleaning, with §3.2.1 missing values and §3.2.2 the binning and smoothing of Section 4.6; §3.5 data transformation and discretisation.
- Müller and Guido, *Introduction to Machine Learning with Python*, O'Reilly, 2016. Ch. 4 on representing data; §6.1–6.2 make the `Pipeline` argument of Section 6. Notebooks [**free**]: `https://github.com/amueller/introduction_to_ml_with_python`
- Wes McKinney, *Python for Data Analysis*, 3e. [**free**] Ch. 7 "Data Cleaning and Preparation" — the pandas half of this lecture: missing data, replacing, `pd.cut`/`pd.qcut`, detecting outliers. `https://wesmckinney.com/book/data-cleaning`
- Stef van Buuren, *Flexible Imputation of Missing Data*, 2e. [**free**] The standard reference, by the author of `mice`. §1.2 MCAR/MAR/MNAR; §1.3.1–1.3.3 listwise deletion and mean imputation, with the variance argument of Section 3.3; §1.3.7 the indicator method; §4.5.2 the MICE algorithm. `https://stefvanbuuren.name/fimd/`
- Kuhn and Johnson, *Feature Engineering and Selection.* [**free**] Ch. 6 Box–Cox and Yeo–Johnson in practice; Ch. 8 missing data. `http://www.feat.engineering/`
- James et al., *An Introduction to Statistical Learning with Python.* [**free**] §3.3.3 outliers and high-leverage points in regression — why Leak C is so seductive. `https://www.statlearning.com/`

Online courses

- **NPTEL, Data Mining** — Prof. Pabitra Mitra, IIT Kharagpur. Lectures 2 and 3 are "Data Preprocessing I" and "II", the closest match to this lecture on NPTEL. [**free**] `https://nptel.ac.in/courses/106105174`
- **NPTEL, Introduction to Machine Learning** — Prof. Balaraman Ravindran, IIT Madras; the reference course for the semester. [**free**] `https://nptel.ac.in/courses/106106139`
- **SWAYAM** — the national portal hosting NPTEL. [**free**] `https://swayam.gov.in/`
- **Kaggle Learn, Data Cleaning** — four hands-on browser lessons (missing values, scaling, dates, inconsistent entry), two or three hours, and a good complement to the Google crash course. [**free**] `https://www.kaggle.com/learn/data-cleaning` `https://developers.google.com/machine-learning/crash-course`

Videos

- Data Mining — IITKGP (NPTEL, Prof. Pabitra Mitra), *Lecture 3 Data Preprocessing - II*: data quality and the preprocessing steps at the board, in the vocabulary you will be examined in. [**free**] `https://youtu.be/wZQM_9vhulg`, after *Lecture 2 Data Preprocessing - I* `https://youtu.be/NSxEiohAH5o`
- Stats with Mia, *Missing Data Assumptions (MCAR, MAR, MNAR)* — the clearest short account of Section 3.4. [**free**] `https://youtu.be/YpqUbirqFxQ`
- AnalytiCode, *Modified Z-Score Explained (Python Outlier Detection)* — exactly the failure of Section 4.3, and the fix. [**free**] `https://youtu.be/wCXQo6OYbhg`

- Phil Chan, *Transforming the response(Y) in regression: Box Cox transformation (7 mins).* [**free**] `https://youtu.be/zYeTyEWt7Cw`

Documentation and interactive explainers

- scikit-learn user guide: *Imputation of missing values* (`SimpleImputer`, `IterativeImputer`, `KNNImputer`, `MissingIndicator`); *Preprocessing data* (`PowerTransformer`, `RobustScaler`); *Pipelines and composite estimators.* [**free**] `https://scikit-learn.org/stable/modules/impute.html`
  `https://scikit-learn.org/stable/modules/preprocessing.html`
  `https://scikit-learn.org/stable/modules/compose.html`

- scikit-learn, *Common pitfalls and recommended practices* — read the section on inconsistent preprocessing before you next report a score. [**free**] `https://scikit-learn.org/stable/common_pitfalls.html`

- Three scikit-learn examples: *Compare the effect of different scalers on data with outliers*; *Map data to a normal distribution*; *Imputing missing values before building an estimator.* [**free**] `https://scikit-learn.org/stable/auto_examples/preprocessing/plot_all_scaling.html`
  `https://scikit-learn.org/stable/auto_examples/preprocessing/plot_map_data_to_normal.html`
  `https://scikit-learn.org/stable/auto_examples/impute/plot_missing_values.html`

- pandas user guide, *Working with missing data*; and `scipy.stats.boxcox` for the maximum-likelihood λ (with `boxcox_llf` for Figure 9). [**free**] `https://pandas.pydata.org/docs/user_guide/missing_data.html`
  `https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.boxcox.html`

Articles

- NIST/SEMATECH, *e-Handbook of Statistical Methods*, §1.3.5.17 "Detection of Outliers" — the authoritative free source for Section 4: the $(n-1)/\sqrt{n}$ ceiling, masking and swamping, and the modified z-score. [**free**] `https://www.itl.nist.gov/div898/handbook/eda/section3/eda35h.htm`

- Brownlee, "Data Leakage in Machine Learning" — a checklist for Section 6. [**free**] `https://machinelearningmastery.com/data-leakage-machine-learning/`

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 3.;Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 3–4.;James, Witten, Hastie, Tibshirani and Taylor, *An Introduction to Statistical Learning with Python*, Ch. 5–6, 12.;Scikit-learn documentation (preprocessing, model selection), accessed 2026-08-04.